

Product Planning & Technical Roadmap

HACK-IT AI

The AI-Powered Hackathon Intelligence Platform

From idea to a pitch-ready presentation, guided by a multi-agent orchestration engine

Prepared by	AC-DC Team
Document type	Internal planning and roadmap dossier
Version	1.0
Date	July 31, 2026
Status	Active development

This document consolidates the platform vision, system design, technology choices, and the phased roadmap that guides the team’s work. It is intended as a shared reference for planning, onboarding, and stakeholder communication.

Contents

1 Executive Summary 3

2 Vision and Objectives 3

3 Product Overview 4

3.1 AI Presentation Generation 4

3.2 AI Hackathon Coach 4

3.3 AI Interview Preparation 4

3.4 AI Workflow Builder 4

3.5 Chat with Presentation 5

3.6 Asynchronous Export and Delivery 5

4 Multi-Agent Orchestration Engine 5

4.1 Agent Responsibilities 5

5 System Architecture 6

5.1 Frontend Composition 7

6 Technology Stack 8

6.1 Frontend 8

6.2 Backend 8

6.3 Infrastructure and Delivery 9

7 Security and Authentication 9

8 Data and Memory Architecture 10

9 Development Roadmap 10

9.1 Milestone Reference Table 11

10 Deployment Strategy 11

11 Team Workflow and Development Setup 12

12	Risks and Mitigations	12
13	Conclusion and Next Steps	13

1 Executive Summary

What we are building, and why it matters.

HACK-IT AI is a platform built to remove the friction between a raw hackathon idea and a finished, presentable project. Teams under time pressure typically lose hours to repetitive work: writing a plan, wiring up a frontend and backend, preparing slides, and rehearsing a pitch. Our platform folds all of that into a guided, AI-assisted pipeline so that participants can spend their limited hours on the parts of the project that actually need human judgement.

At its core, the product is organised around two complementary engines. The first is a **multi-agent orchestration engine** that can take a short natural-language brief and autonomously produce a working full-stack application, complete with a project plan, an architecture document, and a task list. The second is a **presentation and coaching suite** that turns documents or ideas into polished slide decks, offers strategic pitch coaching, and runs voice-based mock interviews. Both engines share a common backend, a common authentication layer, and a common memory system, so that context gathered in one part of the platform can inform another.

The remainder of this document walks through the product vision, the way the system is architected, the technologies we rely on, and — most importantly for planning purposes — the roadmap that sequences our work across the coming phases.

At a Glance

Platform	Web application (Next.js + FastAPI), packaged CLI, and desktop build
Core engines	Multi-agent app builder; AI presentation generator; pitch coach; interview trainer; workflow designer
Primary users	Hackathon participants, student teams, and organisers
Deployment targets	Docker, Kubernetes, Vercel, and Electron desktop

2 Vision and Objectives

The problem we set out to solve and the outcomes we are aiming for.

Hackathons reward speed and clarity, but most of the time available to a team is consumed by mechanical setup rather than genuine problem solving. Our vision is a platform that behaves like an extra teammate: one that can plan an MVP, scaffold the code, catch its own build errors, and prepare the final pitch, while the human team focuses on the idea itself.

We are guided by three objectives that shape every planning decision we make.

- **Reduce time-to-first-demo.** A team should be able to go from a one-line idea to a running, testable application within minutes rather than hours, so that the bulk of a hackathon's timebox is spent iterating rather than bootstrapping.
- **Keep the human in control.** Automation should propose and accelerate, not replace judgement. Every

AI-generated plan, slide outline, or code change is reviewable and editable before it is accepted.

- **Make the final mile easy.** Winning a hackathon is not only about the build; it is about explaining it well. The platform therefore treats the pitch, the demo script, and the judge-facing narrative as first-class outputs, not an afterthought.

These objectives are the lens we use whenever a new feature is proposed: it should either shorten the path to a working build, preserve the team's control over the outcome, or strengthen the final presentation.

3 Product Overview

What the platform actually does, described as a set of connected experiences.

The product is best understood as six connected workflows sitting on top of a shared backend. Each one is streamed to the user in real time using server-sent events, and each one benefits from a persistent memory layer so that the assistant remembers earlier context rather than starting from a blank page every time.

3.1 AI Presentation Generation

This is the flagship workflow. A user uploads supporting documents or simply describes their project, and the system produces a structured outline first, so that the direction can be approved or revised before any slides are drawn. Once approved, the platform generates every individual slide in parallel, applies a chosen visual template, and renders an editable preview. From there, the deck can be exported to PowerPoint or PDF, complete with fonts, images and layout intact.

3.2 AI Hackathon Coach

The coach behaves like an experienced mentor who has read the problem statement and the competition rules. Given a prompt or an uploaded brief, it produces a ranked assessment of possible directions, a comparison of suitable technology stacks, and an hour-by-hour plan mapped against the judging criteria, so a team can make an informed choice quickly rather than debating from first principles.

3.3 AI Interview Preparation

For teams preparing to defend their project to judges, or for participants preparing for a recruiting conversation, this workflow runs a voice-enabled mock interview. Questions are tailored to a role or scenario, answers are analysed as they are spoken, and the system produces a feedback report highlighting strengths and gaps, together with sensible follow-up questions.

3.4 AI Workflow Builder

This is a visual, drag-and-drop canvas for designing multi-agent automation pipelines. A short natural-language description is turned into both a diagram and a working set of canvas nodes, and the two stay in sync: editing the diagram updates the canvas, and rearranging the canvas updates the diagram.

3.5 Chat with Presentation

Once a deck exists, a user can simply ask for changes in plain language, for example asking that a particular slide be made more concise. The assistant has access to the full slide content, the conversation history, and long-term memory of the user's preferences, so edits tend to land correctly on the first try.

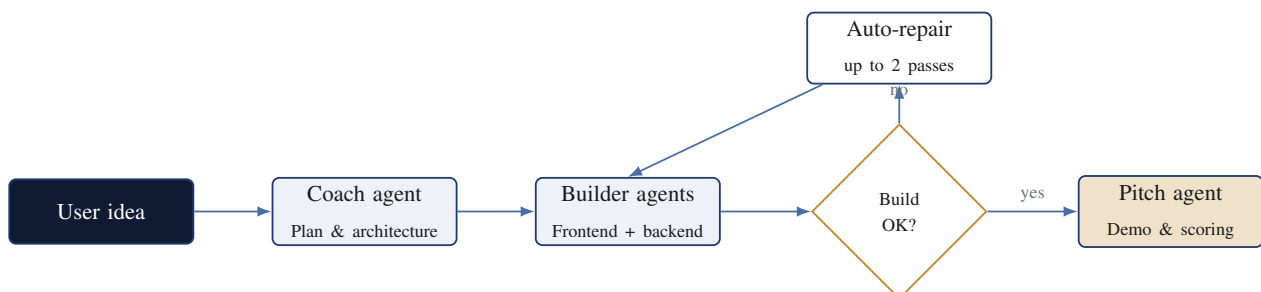
3.6 Asynchronous Export and Delivery

Because rendering a full slide deck to PowerPoint or PDF can take real time, exports are handled as background jobs. The user receives an immediate acknowledgement, the frontend polls for completion, and — for integrations that need it — a signed webhook notifies external systems the moment the file is ready.

4 Multi-Agent Orchestration Engine

How a single idea becomes a working, self-repairing, pitch-ready project.

The orchestration engine is the most ambitious piece of the platform. It runs as its own terminal application and coordinates six specialised agents, each responsible for a distinct phase of building a project. What makes the pipeline reliable is not any single agent, but the sequencing: planning happens before building, building happens before validation, and validation is allowed to loop back and repair itself before the pitch materials are ever generated.



The Coach agent is the planner: it turns a short prompt into an MVP scope, a component and data architecture, and a checkbox task list. The two Builder agents then work in parallel, one producing the frontend and the other the backend, so that both halves of the application are ready at roughly the same time rather than one blocking the other. Every build is verified automatically; if the build fails, the Auto-Repair agent inspects the logs and patches the source, and it is allowed up to two attempts before the pipeline escalates the issue. Only once a build passes does the Pitch agent take over, producing a three-minute demo script and a rubric-based self-assessment that gives the team an honest preview of how a judge might score the submission.

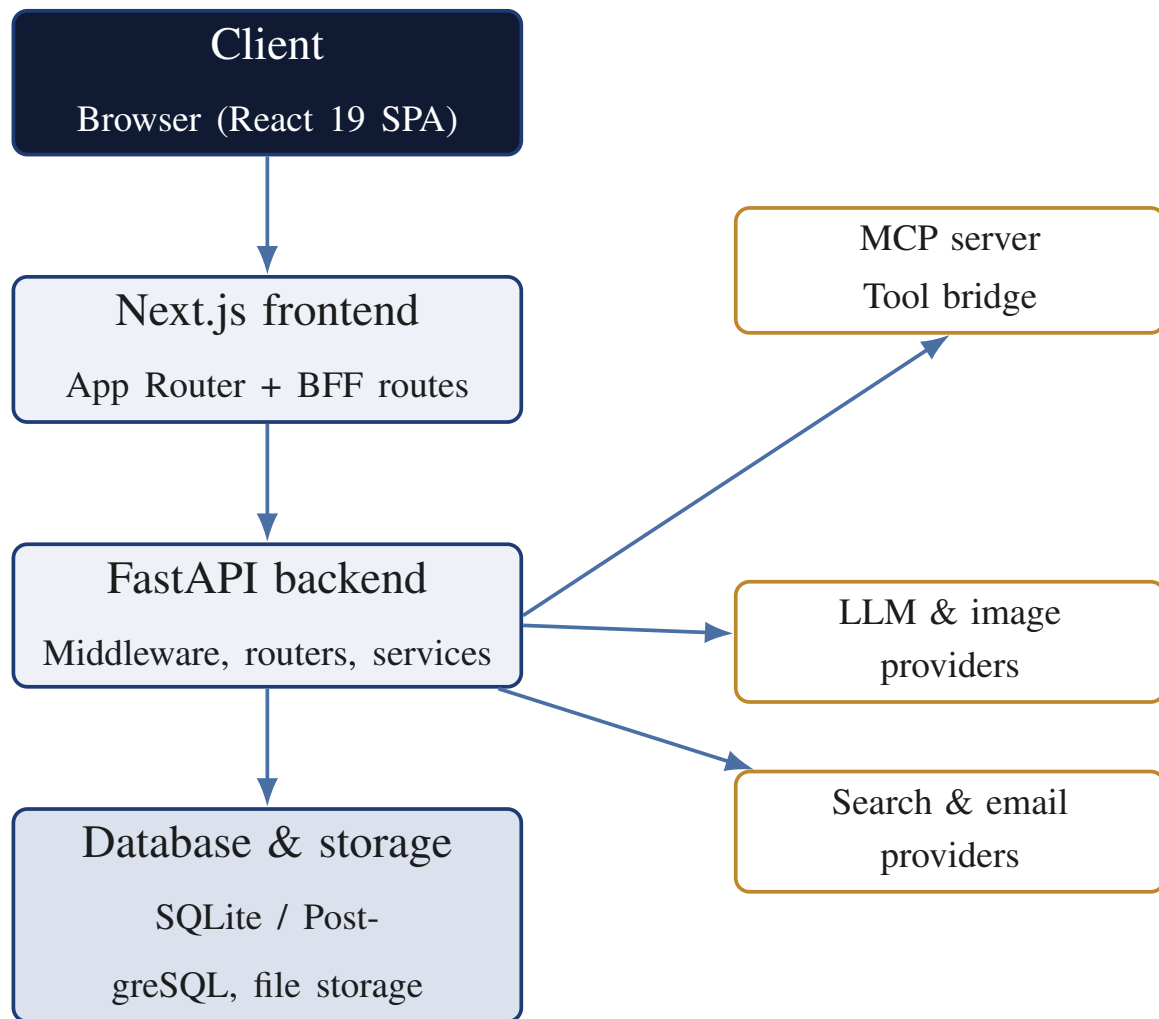
4.1 Agent Responsibilities

Agent	Role	Primary output
Coach	Scope definition and architecture design	Project plan, architecture document, task list
Frontend builder	Generates the complete client application	A working React and Vite interface with a consistent design system
Backend builder	Generates the complete server application	A Node.js and Express REST API backed by SQLite
Validation agent	Runs the build, lint, and test suite	Structured error reports feeding into repair instructions
Auto-repair agent	Applies targeted fixes and re-runs validation	Patched source files, bounded to two iterations
Pitch agent	Prepares the team's final narrative	Demo script and judge-facing self-score

5 System Architecture

How the pieces of the platform fit together end to end.

The platform is organised as a conventional but carefully layered web application. A Next.js frontend serves the user interface and also acts as a backend-for-frontend, proxying requests through to a FastAPI service that owns the business logic, the database, and every integration with external AI providers. An optional MCP server exposes the same backend capabilities as callable tools, so that other AI systems can drive the platform programmatically rather than through the browser alone.



Two design decisions are worth calling out because they shape how we plan future work. First, the frontend never talks to external AI providers directly; every model call is mediated by the backend, which keeps API keys off the client and gives us one place to add logging, rate limiting, or provider failover. Second, the database layer is deliberately swappable: SQLite is used for local development so that a new contributor can be productive within minutes, while PostgreSQL is used in production for the concurrency and durability guarantees a live deployment needs.

5.1 Frontend Composition

The frontend is structured around a root layout that supplies fonts, smooth scrolling, and global state, with three broad areas built on top of it: a marketing landing page, an authenticated presentation generator (upload, outline review, and the slide editor itself), and a set of standalone feature modules for the coach, the interview trainer, the hackathon browser, and the workflow builder. Application state that needs to persist across these areas — the live generation stream, the upload queue, user configuration, and editor undo history — lives in a small set of Redux slices rather than being scattered across components.

6 Technology Stack

The tools we rely on, and why each one earned its place.

We keep the stack intentionally boring where reliability matters and intentionally modern where user experience benefits from it. The frontend favours a fast, component-driven React setup; the backend favours a typed, async-first Python framework with a broad bench of AI provider integrations so that no single vendor is a single point of failure.

6.1 Frontend

Category	Technology	Why it is used
Framework	Next.js 16 with the App Router	Server components, routing, and backend-for-frontend handlers in one project
UI library	React 19	Component model shared across the whole client
Language	TypeScript	Type safety across a large, fast-moving codebase
Styling	Tailwind CSS 4	Rapid, consistent styling without hand-written CSS sprawl
State management	Redux Toolkit	Predictable state for streaming generation and editor history
Motion	Framer Motion	Polished, purposeful interface animation
Rich editing	TipTap, dnd-kit, Monaco Editor	Slide text editing, drag-and-drop re-ordering, and code/layout editing

6.2 Backend

Category	Technology	Why it is used
Framework	FastAPI on Uvicorn	High-throughput async API with automatic documentation
ORM & migrations	SQLModel with Alembic	Typed database models and versioned schema changes
Database	SQLite in development, PostgreSQL or MySQL in production	Fast local iteration with a durable production path
AI integration	Fifteen LLM providers via a common adapter layer	Redundancy and the freedom to pick the best model per task
Vector memory	FastEmbed with Mem0	Local semantic search and persistent conversational memory

Category	Technology	Why it is used
Document parsing	pdfplumber, python-pptx, Pillow	Ingesting uploaded briefs and existing decks
Tool bridge	FastMCP	Exposes backend capabilities as callable tools for other AI agents
Authentication	Custom PBKDF2-HMAC-SHA256 implementation	Full control over the security model with no external dependency

6.3 Infrastructure and Delivery

Tool	Purpose
Docker and Compose	Local and single-host containerised deployment
Kubernetes	Production-grade, horizontally scalable deployment
Nginx	Reverse proxy in front of the frontend and backend
Vercel	Frontend-only deployment on a global edge network
Electron	Packaging the platform as a self-contained desktop application

7 Security and Authentication

How we protect user accounts and sessions.

Rather than depending on a third-party authentication service, the team built the authentication layer from scratch, which gives us full visibility into how credentials are handled and keeps the platform easy to self-host without external accounts. A user’s password is never stored directly; it is hashed with PBKDF2-HMAC-SHA256 using two hundred thousand iterations, which meets the current NIST-recommended minimum, and every comparison is performed in constant time to guard against timing attacks. Once a user is signed in, their session is represented by a compact, HMAC-signed token carried in an HTTP-only cookie, valid for thirty days and rejected automatically once it expires.

For accounts that want an additional layer of protection, the platform supports time-based one-time-password two-factor authentication. A user scans a generated QR code into an authenticator app, and from that point on every login is checked against a fresh six-digit code before a session is issued. New sign-ups can also be verified by email using a one-time code, which closes off a common source of throwaway or spoofed accounts.

Security Summary	
Password hashing	PBKDF2-HMAC-SHA256, 200,000 iterations, per-user random salt
Session tokens	HMAC-SHA256 signed, thirty-day expiry, HTTP-only cookie or bearer header
Two-factor authentication	TOTP via an authenticator app, with QR-code enrolment
Email verification	Six-digit one-time code delivered on sign-up

8 Data and Memory Architecture

Where information lives, and how the assistant remembers it.

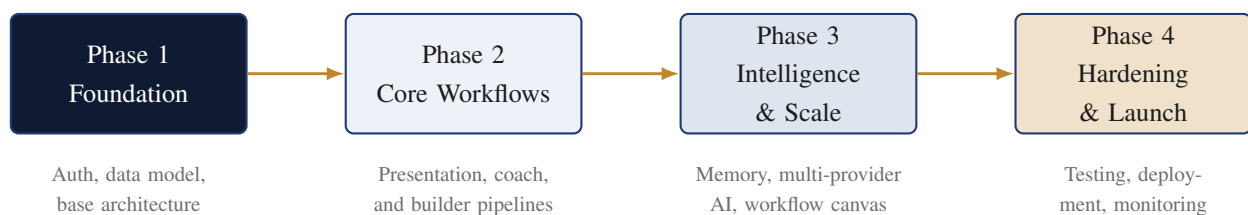
The relational database is the system of record for everything a user can see: presentations, slides, chat history, uploaded documents, async export jobs, and account data. On top of that sits a separate long-term memory store, built on Mem0, which is not a copy of the database but a distillation of it — a running summary of what the assistant has learned about a user’s preferences and past projects, so that a returning user does not have to re-explain context that was already established. Semantic search across a user’s own documents is handled locally with FastEmbed, which avoids sending document content to a third-party vector database and keeps search latency low.

In practice this means every AI call in the platform — whether it is generating an outline, coaching on a problem statement, or editing a slide — is composed from three ingredients: the immediate request, the relevant slice of database history, and the relevant slice of long-term memory. That composition is what makes the assistant feel consistent across a session rather than stateless.

9 Development Roadmap

How the work is sequenced, and what defines progress at each stage.

Planning for HACK-IT AI is organised into four phases. Each phase has a clear exit criterion, which keeps the roadmap honest: a phase is complete when its criterion is met, not when a calendar date passes. The diagram below gives the overall shape of the sequence, and the table that follows it breaks each phase into the concrete work it contains.



Phase 1 — Foundation

This phase establishes everything later phases depend on: the authentication system, the core relational schema, and the base Next.js and FastAPI project skeleton with the proxy layer connecting them. Its exit criterion is a deployable, empty shell of the platform where a user can register, sign in, and reach an authenticated page.

Phase 2 — Core Workflows

With the foundation in place, this phase builds the features users actually came for: document upload and parsing, the outline-then-slide generation pipeline, the hackathon coach, and the first working version of the multi-agent app builder. Its exit criterion is that a user can go from a prompt to an exported

presentation, and separately, from a prompt to a scaffolded application, without manual intervention.

Phase 3 — Intelligence and Scale

This phase deepens the platform rather than widening it. Long-term memory is introduced so the assistant improves with repeated use, additional LLM, image, and search providers are integrated for redundancy and choice, and the visual workflow builder and voice-based interview trainer are added. Its exit criterion is that the platform can gracefully continue functioning even if any single external provider becomes unavailable.

Phase 4 — Hardening and Launch

The final phase is about trustworthiness rather than new features: expanding automated test coverage, finalising the Docker and Kubernetes deployment paths, wiring up production monitoring through Sentry and analytics through Mixpanel, and packaging the Electron desktop build. Its exit criterion is a production deployment that the team is confident handing to real users without close supervision.

9.1 Milestone Reference Table

Phase	Key deliverables	Exit signal
Foundation	Custom auth with 2FA, database schema, Next.js/FastAPI skeleton	Authenticated shell is deployable
Core Workflows	Presentation generator, coach, first app-builder pipeline	Idea-to-export works end to end
Intelligence & Scale	Mem0 memory, multi-provider AI, workflow builder, interview trainer	Provider outages degrade gracefully
Hardening & Launch	Test coverage, Docker/K8s, monitoring, Electron packaging	Ready for unsupervised real-world use

10 Deployment Strategy

How the platform reaches its users across different environments.

Rather than committing to a single deployment target, the platform is built to run comfortably in four different contexts, which matters because our users range from a solo hackathon organiser running things locally to a team deploying at scale.

- **Docker Compose** is the default path for anyone standing up the full stack quickly, whether for local development or a small self-hosted instance. A single command brings up the frontend, backend, and database together.

- **Kubernetes** is the production path for deployments that need to scale horizontally, with an ingress controller in front of pod replicas of the frontend and backend, and a stateful PostgreSQL deployment backed by persistent storage.
- **Vercel** allows the frontend to be deployed independently on a global edge network when the backend is hosted elsewhere, which is useful for teams that want the fastest possible page loads.
- **Electron** packages the entire platform, backend included, as a self-contained desktop application talking to a local SQLite database — well suited to offline use or environments where installing a server stack is impractical.

11 Team Workflow and Development Setup

How the team gets from a fresh checkout to a running instance.

New contributors are expected to be productive quickly. The backend is brought up with a single dependency sync and a reload-enabled development server, with interactive API documentation available immediately. The frontend follows the familiar install-and-run pattern for a Next.js project, pointed at the local backend through one environment variable. The optional app-builder CLI can be run either from source or installed instantly through npm, which keeps the barrier to trying the multi-agent pipeline as low as possible.

Every backend change is expected to pass the automated test suite before merging, and schema changes are made exclusively through Alembic migrations rather than manual edits, so that the database history stays reproducible across every environment the platform runs in.

Everyday Commands

Backend (dev server)	<code>uv sync && uv run python server.py -port 8000 -reload true</code>
Frontend (dev server)	<code>npm install && npm run dev</code>
Run tests with coverage	<code>uv run pytest -cov</code>
Apply migrations	<code>alembic upgrade head</code>
Full stack via Docker	<code>docker compose up -d</code>

12 Risks and Mitigations

What could slow us down, and how the plan already accounts for it.

No roadmap is complete without an honest look at what might go wrong. The risks below are the ones we track most closely, together with how the current design already reduces each one.

- **Dependence on external AI providers.** A single provider outage or rate limit could stall every AI-driven workflow. This is mitigated by supporting fifteen interchangeable LLM providers behind one adapter layer, so traffic can shift away from a struggling provider without a code change.
- **Build and repair loops that never converge.** An automatically generated project could, in principle,

fail to build indefinitely. The auto-repair agent is deliberately capped at two attempts, after which the pipeline surfaces the failure rather than looping silently.

- **Scope creep across six workflows.** With six distinct AI workflows in active development, it is easy for effort to spread too thin. The phased roadmap exists specifically to enforce sequencing, so that core workflows are solid before the team invests further in scale and polish.
- **Security of a hand-built authentication system.** Building authentication in-house avoids third-party dependency but places the responsibility for its correctness entirely on the team. This is why the implementation follows well-established primitives, PBKDF2 with NIST-recommended iteration counts and constant-time comparison, rather than anything bespoke at the cryptographic level.

13 Conclusion and Next Steps

Where things stand, and what the team focuses on next.

HACK-IT AI already covers the full arc from a raw idea to a pitch-ready project: planning, building, presenting, and coaching are all represented in working workflows today. The priority ahead is less about adding new surface area and more about deepening what already exists, strengthening the memory layer, broadening provider redundancy, and finishing the hardening work that turns a capable prototype into a platform the team is comfortable putting in front of real hackathon teams under real time pressure.

This document should be treated as a living reference. As phases close and priorities shift, the roadmap in Section 9 is the section to revisit and update first, since it is the one the rest of the team's planning depends on most directly.

Built for hackathons. Designed for winners. — AC-DC Team